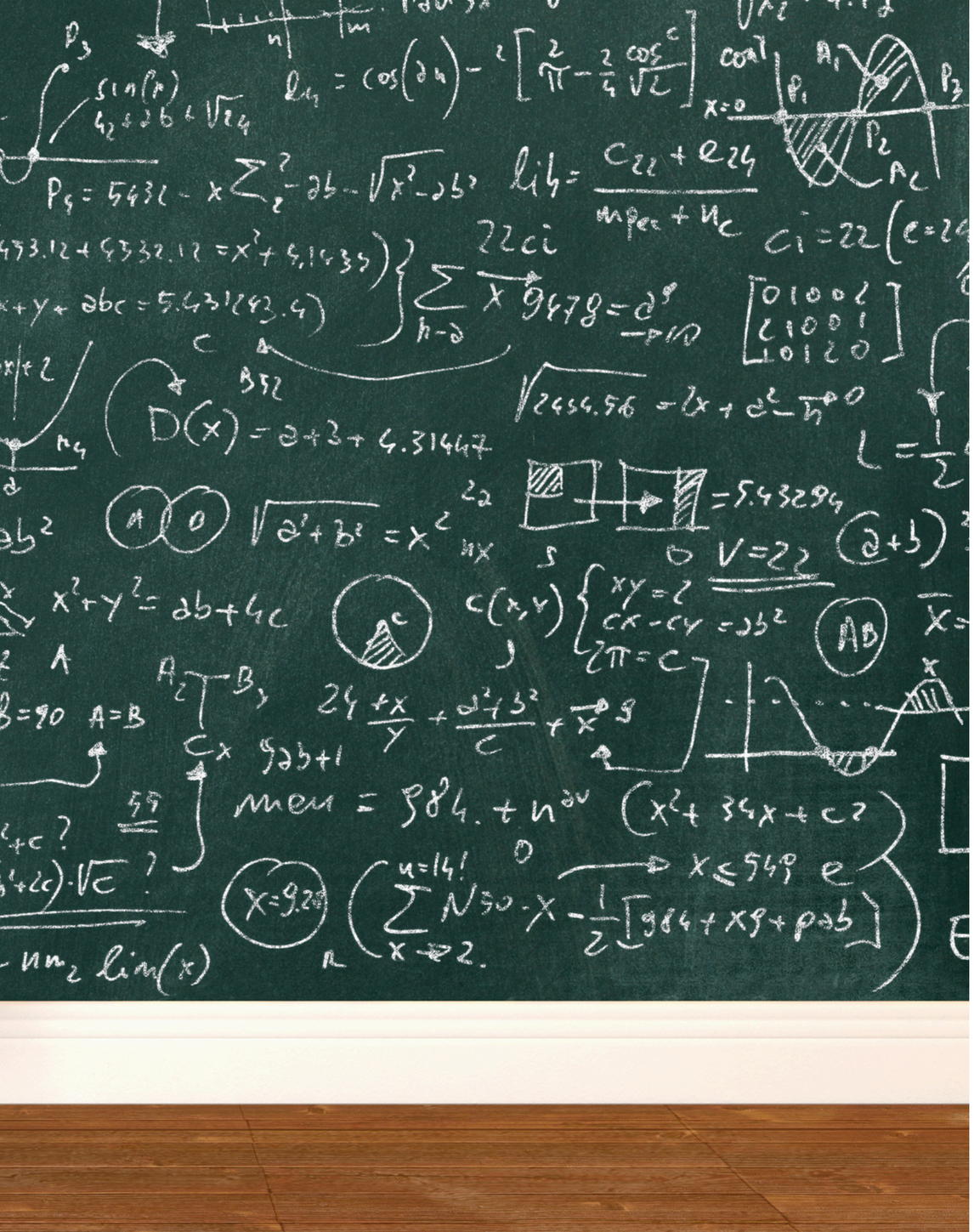


Diagramy klas

...czyli prosty język opisu modelu obiektowego

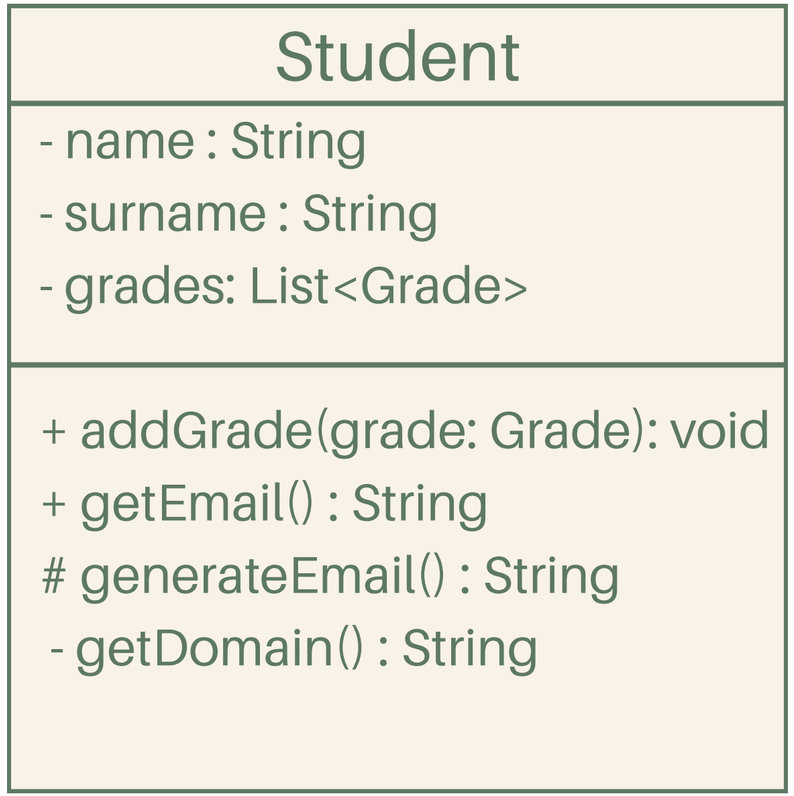
Michał Idzik, WI AGH

UML ?

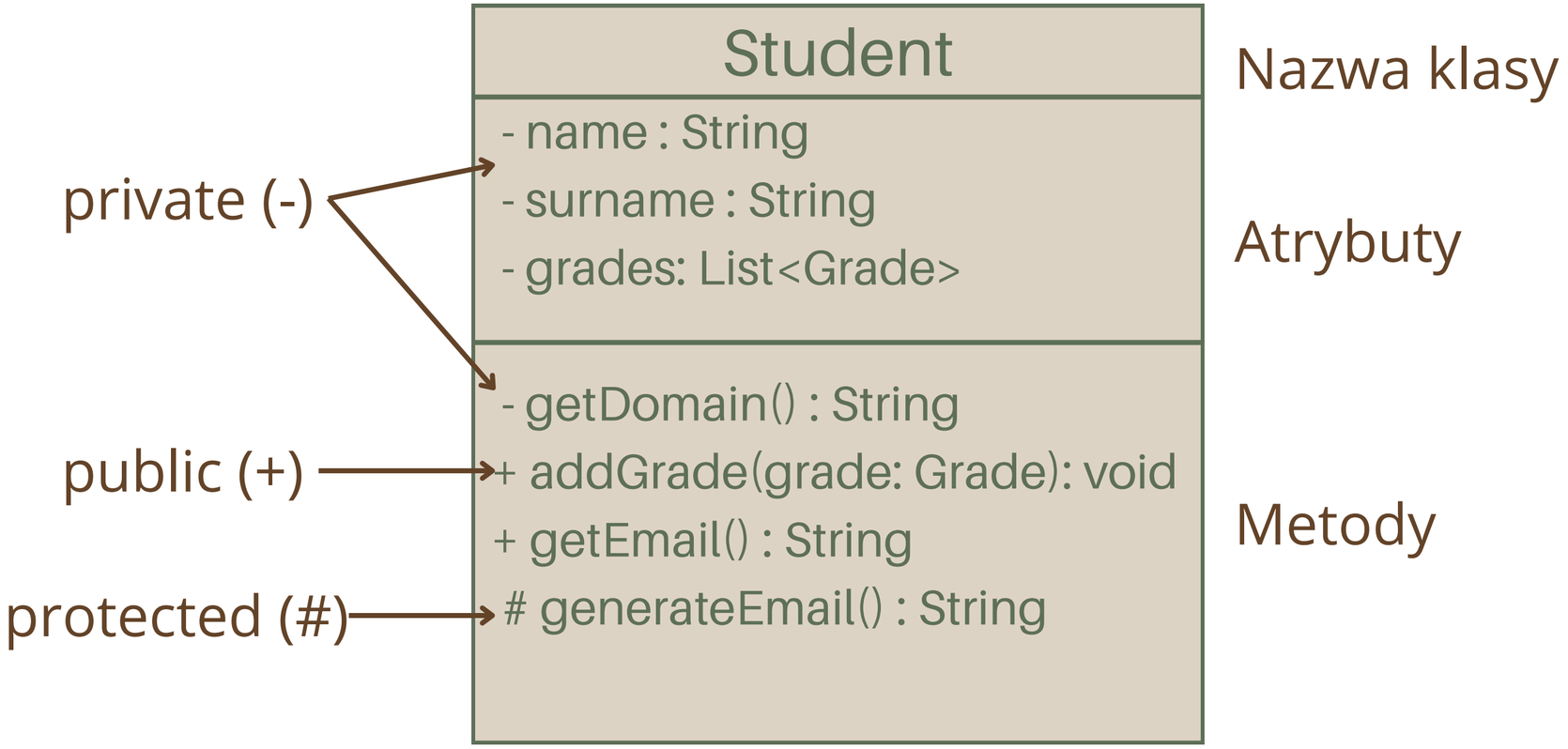
Unified **M**odeling **L**anguage

- Graficzny język opisu modelu logicznego (struktury, przepływy danych itp)
- Wiele rodzajów diagramów, np. diagramy klas
- Jeden spójny standard opisu = brak nieporozumień

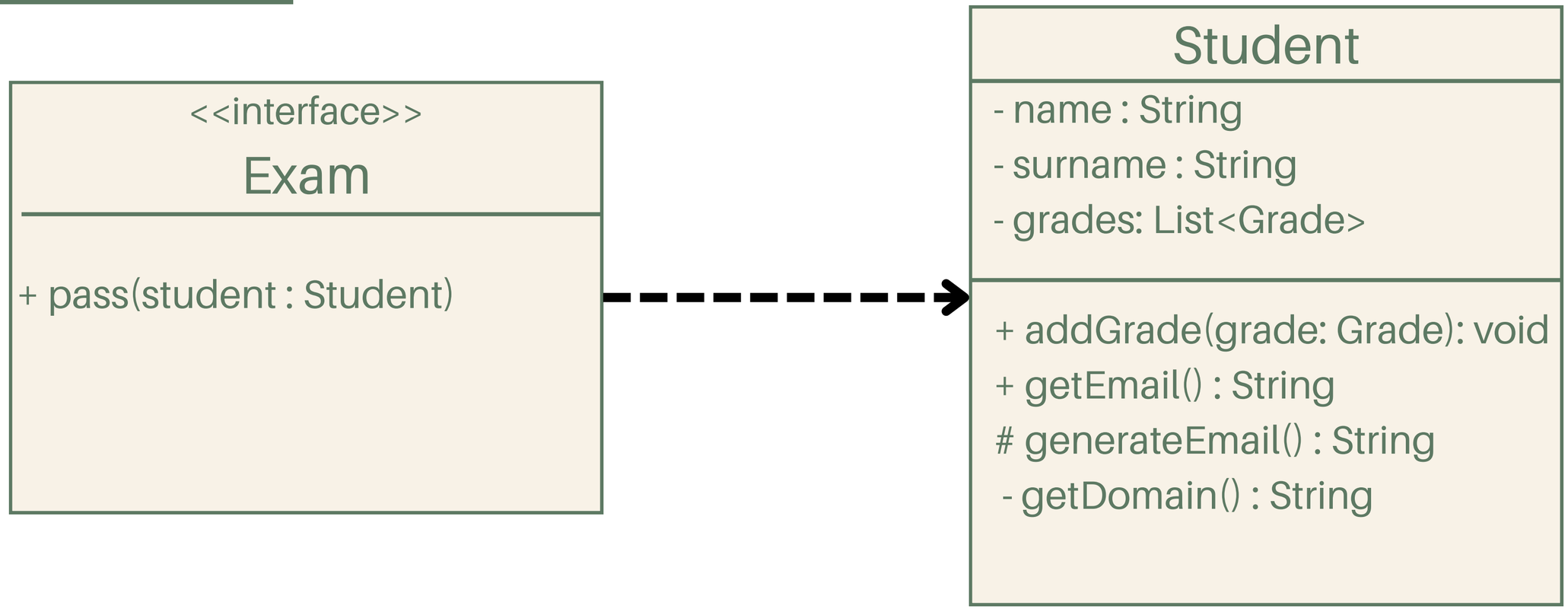
Definicja klasy



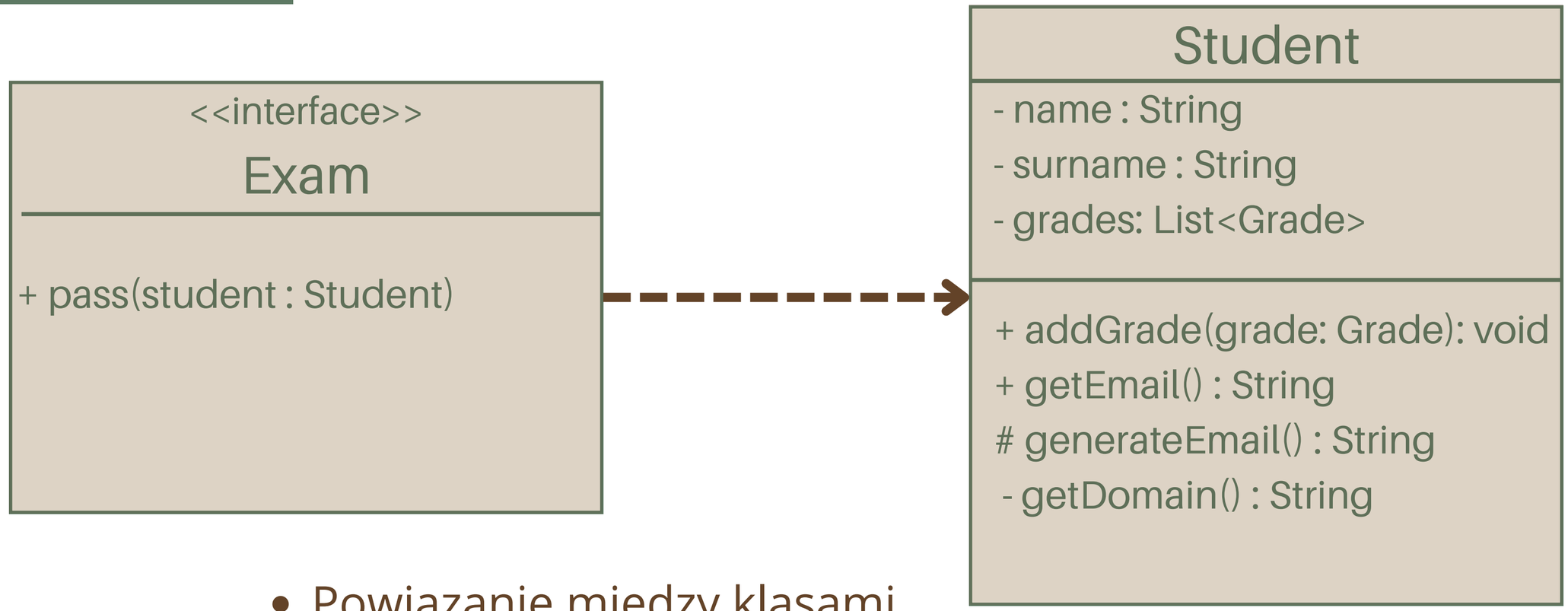
Definicja klasy



Relacja

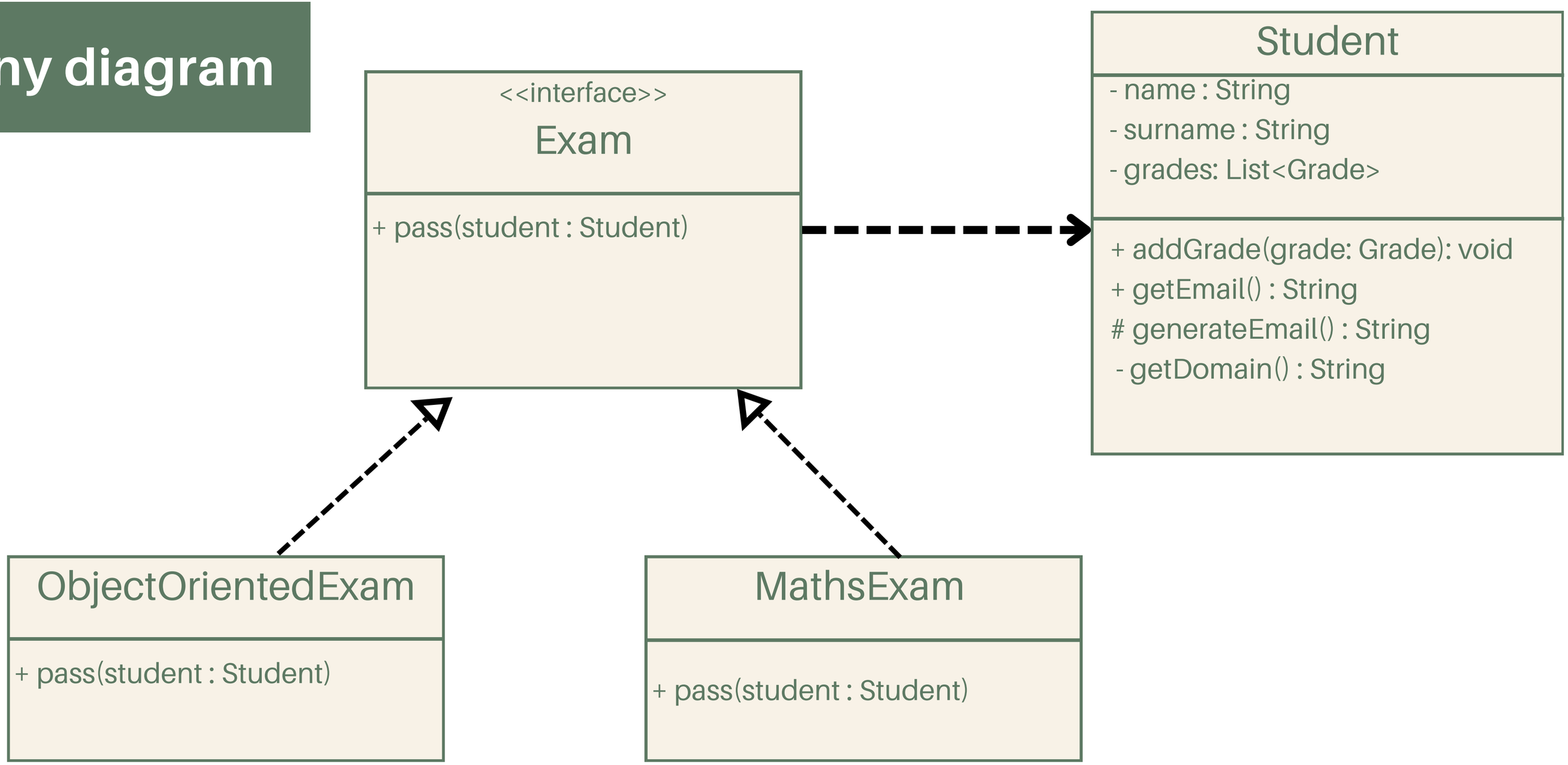

<<interface>>
Exam
+ pass(student : Student)
Student
- name : String
- surname : String
- grades: List<Grade>
+ addGrade(grade: Grade): void
+ getEmail() : String
# generateEmail() : String
- getDomain() : String

Relacja



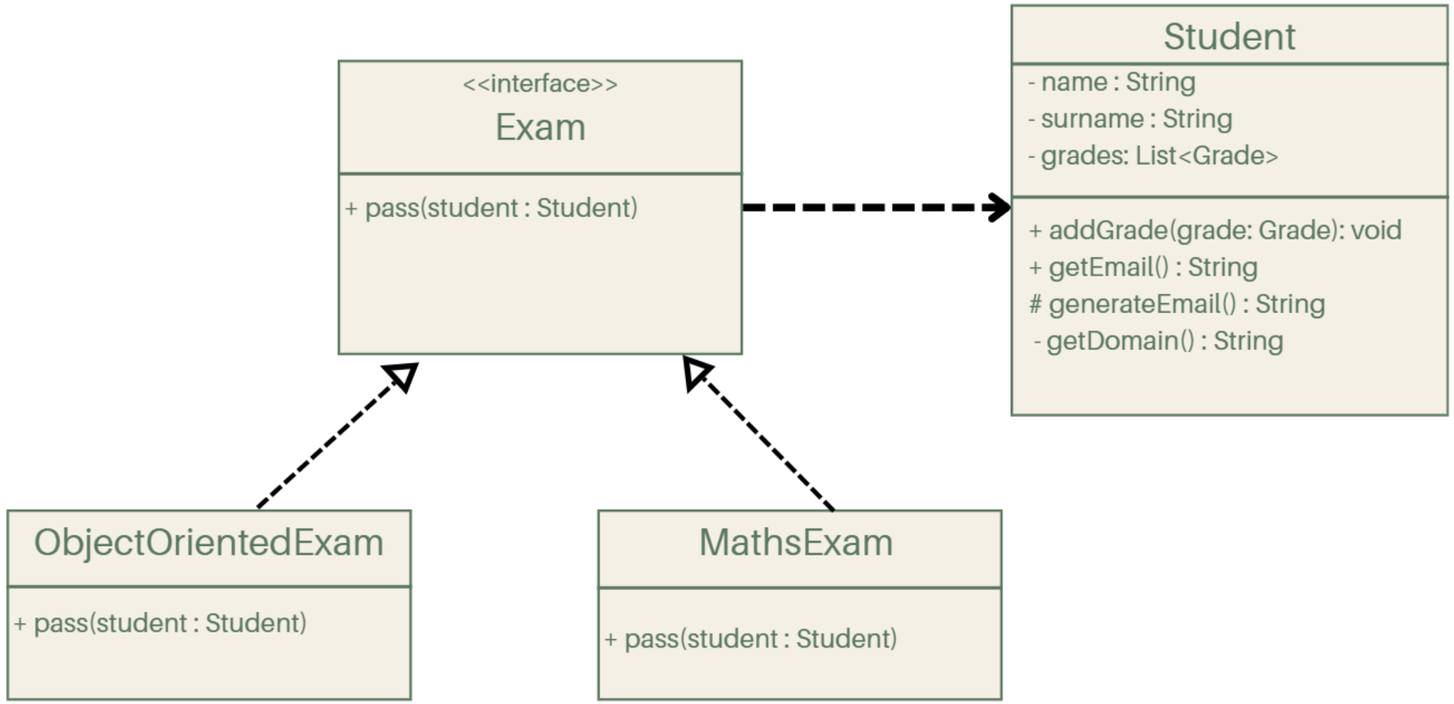
- Powiązanie między klasami
- Może posiadać określony zwrot (lub dwustronne)
- Wiele różnych rodzajów relacji

Pełny diagram

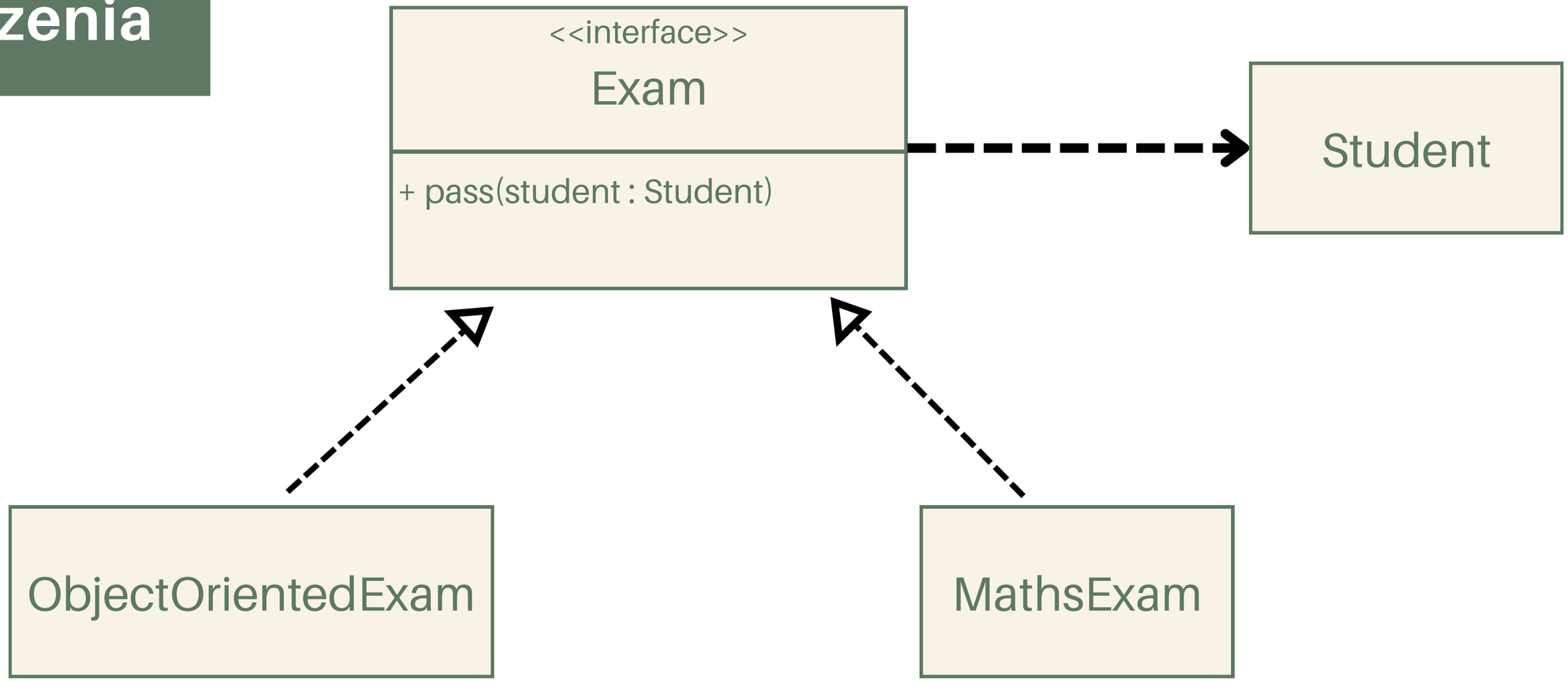


Pełny diagram

- Interfejs *Exam* jest realizowany przez dwie klasy (*ObjectOrientedExam*, *MathsExam*)
- *Exam* posiada metodę *pass()*, która używa obiektu klasy *Student*



Uproszczenia



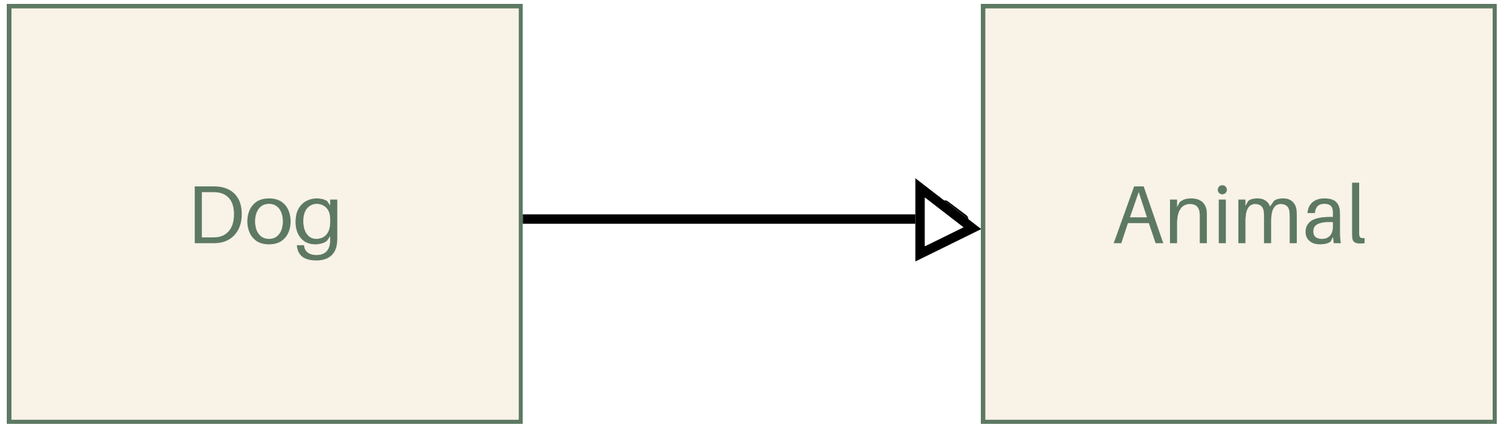
Możemy pominąć deklaracje atrybutów i metod, jeśli zależy nam na pokazaniu np. samej struktury (poziomy abstrakcji!)

Rodzaje relacji

Dziedziczenie

- reprezentuje relację generalizacji dla klas
- Klasa **A** rozszerza (dziedziczy) klasę **B**

Dziedziczenie



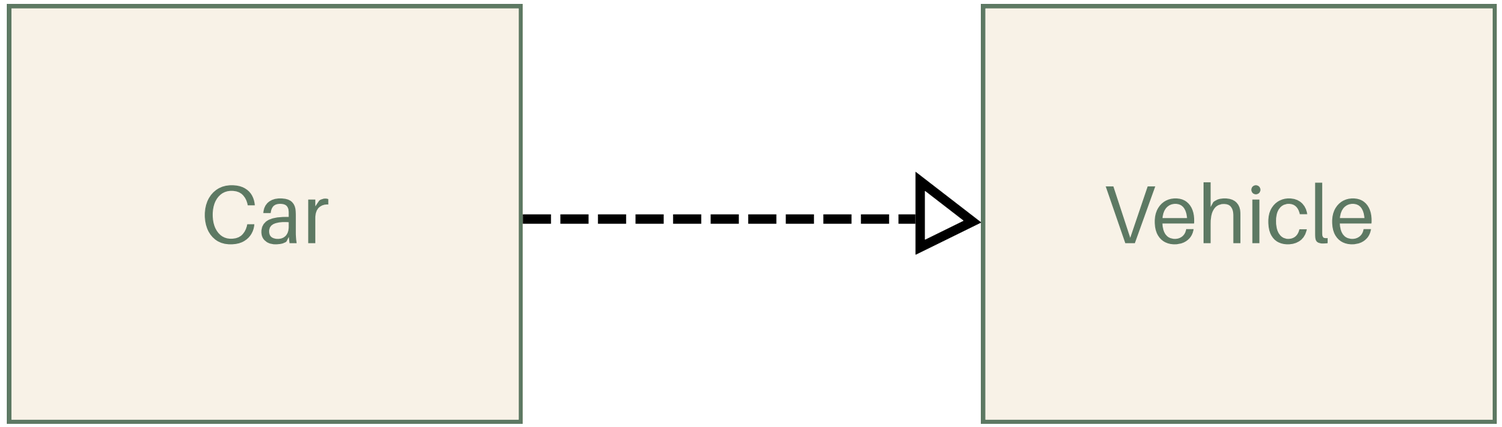
“Pies jest zwierzęciem“

Implementacja

- reprezentuje relację generalizacji poprzez interfejs
- Klasa **A** implementuje interfejs **B**

“potrafi”

Implementacja



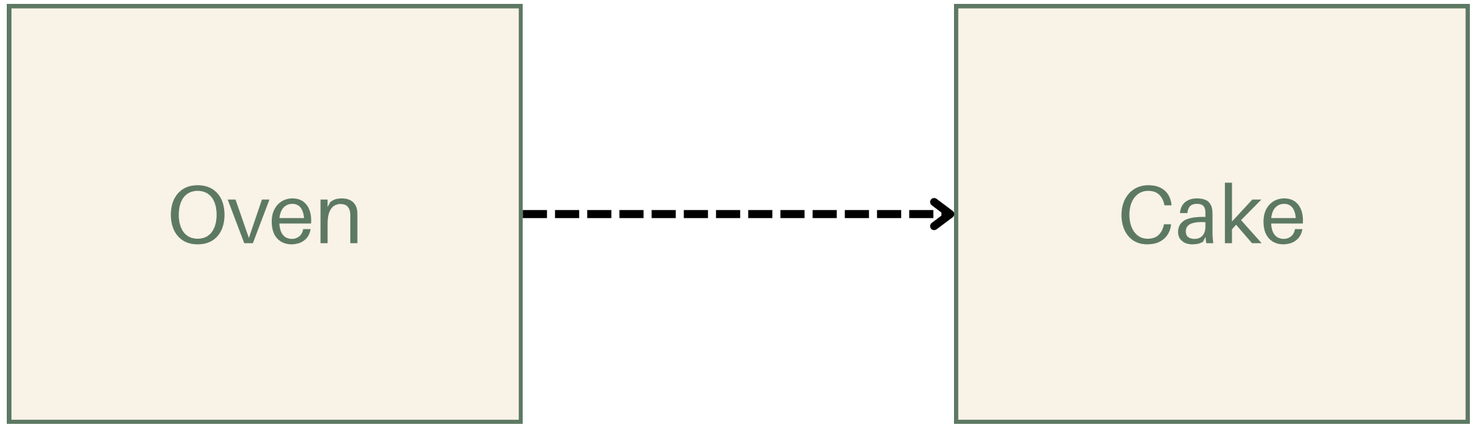
“Samochód jest pojazdem, więc potrafi jeździć”

Zależność

- reprezentuje tymczasowe powiązanie między klasami
- najczęściej: klasa **A** przyjmuje obiekt **B** jako parametr metody (ale nie zapisuje go jako atrybut)

“używa”

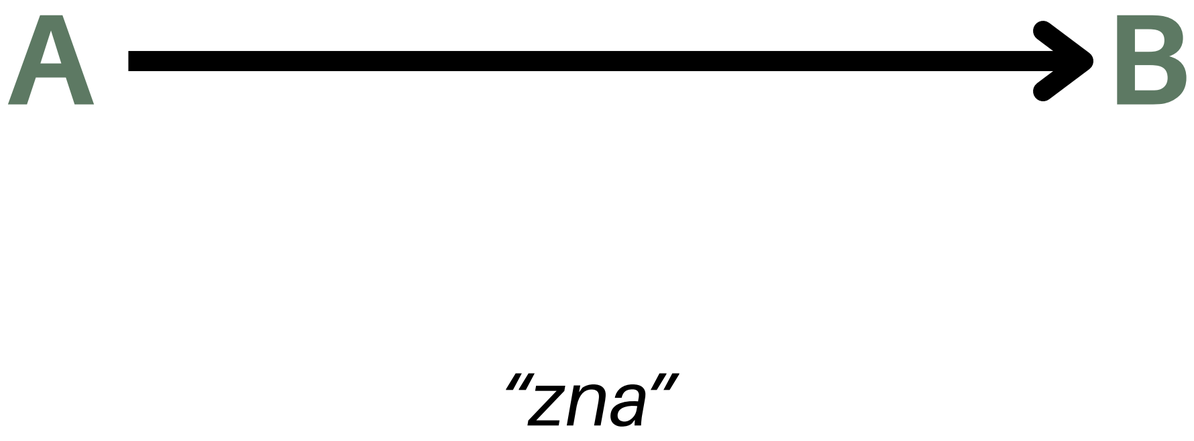
Zależność



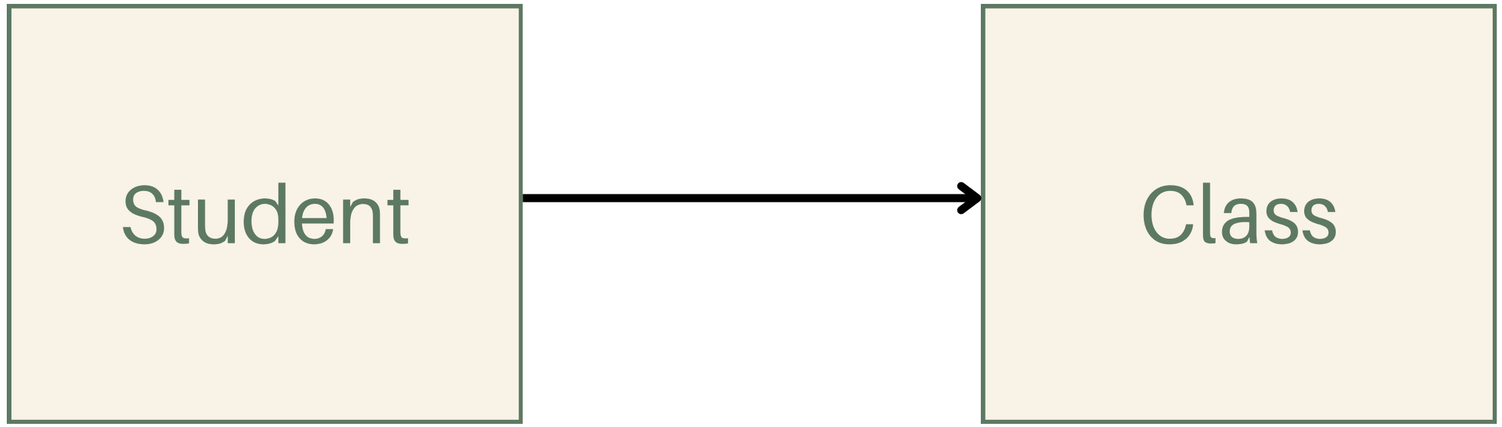
“Piekarnik używa ciasta tylko w momencie jego pieczenia. Upieczone ciasto od razu wyjmujemy z piekarnika”.

Asocjacja

- reprezentuje powiązanie między klasami
- bardzo ogólna, obiekty klas mogą istnieć niezależnie
- najczęściej: klasa **B** jest atrybutem klasy **A**



Asocjacja



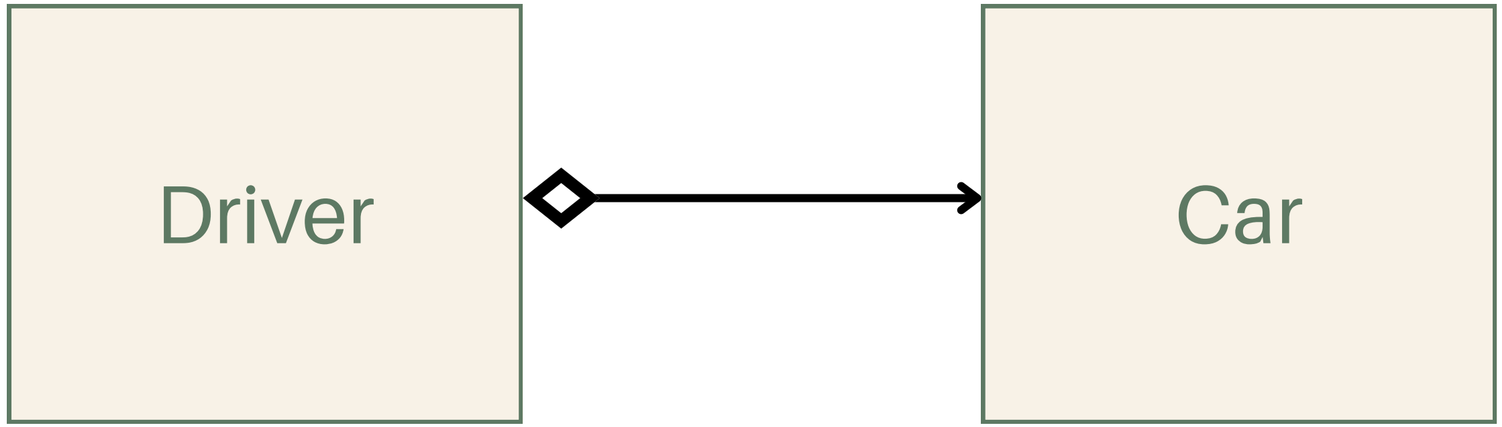
“Student zna zajęcia, na które ma chodzić i może w każdej chwili z tej wiedzy skorzystać”

Agregacja

- silniejsza forma asocjacji
- obiekt klasy **B** **jest częścią** **A**
- obiekt podrzędny nadal może istnieć niezależnie
- najczęściej stosujemy przy atrybutach-kolekcjach (ale nie tylko!)

“posiada”

Agregacja



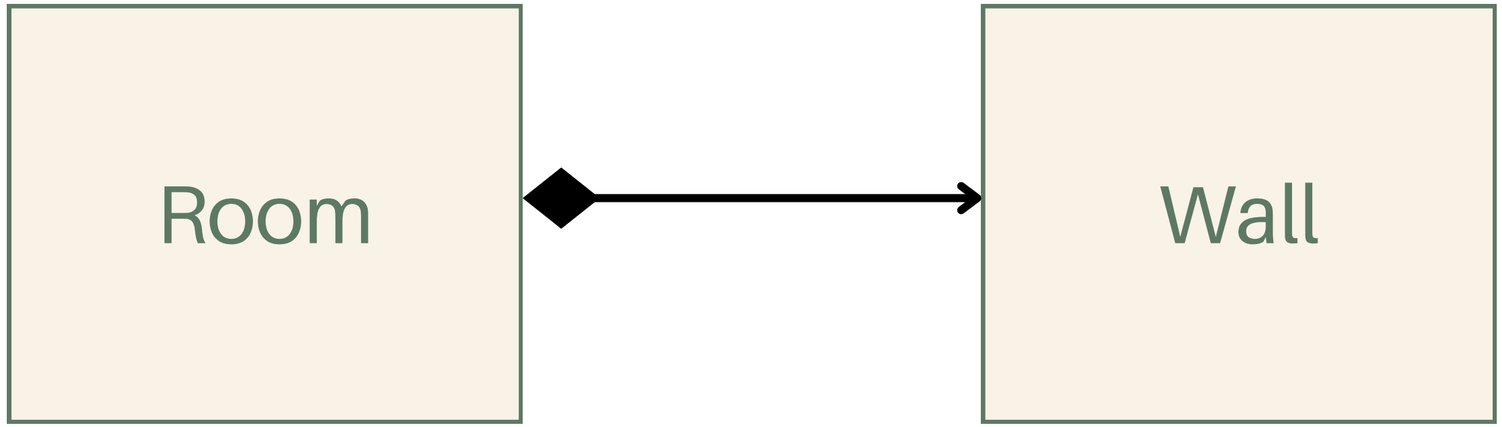
“Kierowca posiada samochód.
Samochód ten nadal może zmienić właściciela”

Kompozycja

- silniejsza agregacja
- obiekt klasy **B** **jest częścią** **A** i **nie może istnieć bez niego**

“składa się z..”

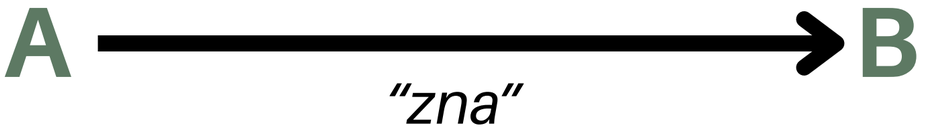
Kompozycja



“Pokój składa się z ścian.
Jeśli usuniemy pokój, nie będzie też ścian.”

Dziedziczenie

Asocjacja

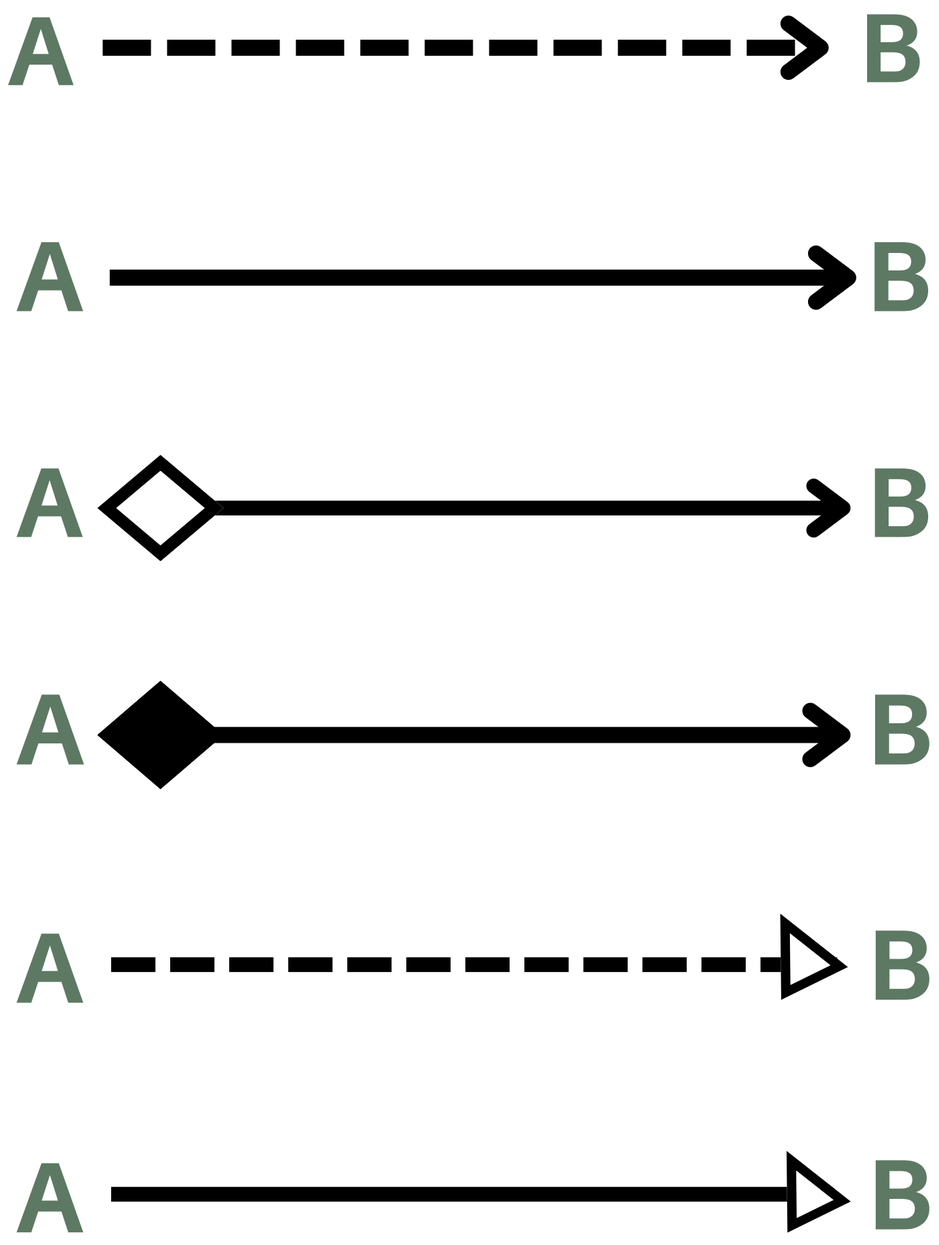


Implementacja

Agregacja

Zależność

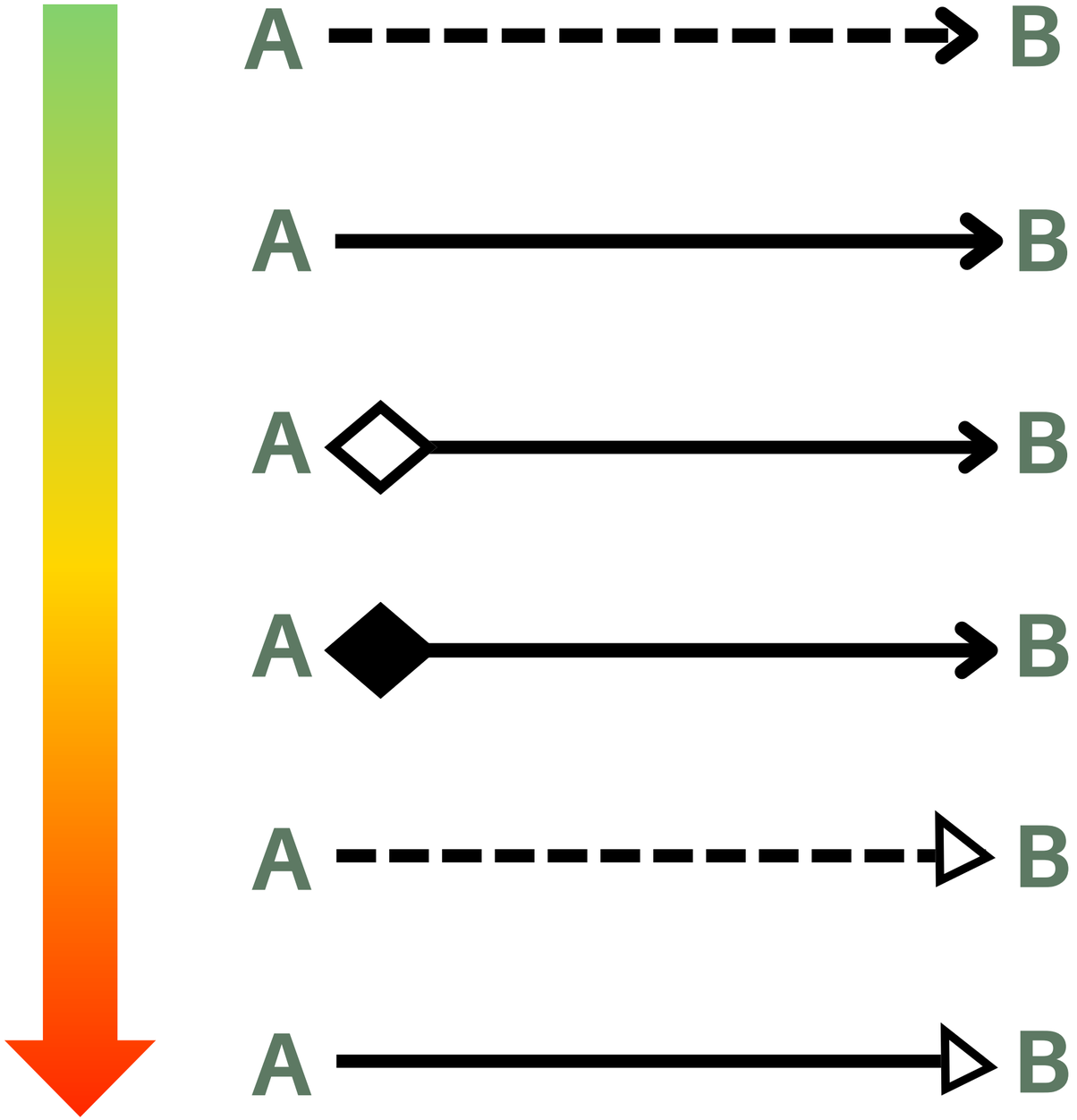
Kompozycja


A
B
A
B
A
B
A
B
A
B
A
B

Siła relacji

Relacje można posortować od najsłabszej (zależność) do najsilniejszej (dziedziczenie).

Im silniejsza relacja tym większe **sprzężenie** (*coupling*).
Większe sprzężenie = trudniejszy do rozszerzenia kod.



Więcej szczegółów → materiały dodatkowe (GitHub)

Zadanie: **Obiektowe Kalambury**